

Sales Performance Analysis of Walmart Stores Using Advanced SQL Techniques

Student Details

Name: B. Varshitha

Abstract

This project analyzes Walmart sales data using advanced SQL techniques. The objective is to identify sales trends, customer behavior, product profitability, and payment preferences. The analysis provides business insights that help improve business decision-making.

Tools and Technologies Used

- Google Colab
- Python (Pandas)
- SQLite
- SQL
- Microsoft Word
- GitHub

Dataset Description

The Walmart sales dataset contains transaction-level data with the following fields: Invoice ID, Branch, City, Customer Type, Gender, Product Line, Unit Price, Quantity, Tax, Total, Date, Time, Payment, COGS, Gross Income, and Rating.

Task 1: Identifying the Top Branch by Sales Growth Rate

Objective

To identify the branch with the highest average monthly sales growth.

SQL Query

```
WITH monthly_sales AS (  
SELECT  
Branch AS branch,  
strftime('%Y-%m', Date) AS month,  
SUM(Total) AS total_sales  
FROM walmart_sales  
GROUP BY Branch, month  
)  
sales_growth AS (  
SELECT  
branch,
```

```

month,
total_sales,
LAG(total_sales) OVER (
PARTITION BY branch
ORDER BY month
) AS previous_month_sales
FROM monthly_sales
)
SELECT
branch,
ROUND(
AVG(
CASE
WHEN previous_month_sales IS NOT NULL
AND previous_month_sales <> 0
THEN ((total_sales - previous_month_sales) / previous_month_sales) * 100
END
),
2
) AS avg_growth_rate
FROM sales_growth
GROUP BY branch
ORDER BY avg_growth_rate DESC;

```

Result

	branch	avg_growth_rate
0	B	17.98
1	C	5.96
2	A	-3.28

Explanation

This query calculates monthly sales for each branch and determines the average growth rate compared with the previous month.

Task 2: Finding the Most Profitable Product Line for Each Branch

Objective

To determine the most profitable product line in each branch.

SQL Query

```
WITH product_profit AS (  
  SELECT  
    Branch AS branch,  
    "Product line" AS product_line,  
    SUM("Gross income" - cogs) AS profit  
  FROM walmart_sales  
  GROUP BY Branch, "Product line"  
)  
,  
ranked_profit AS (  
  SELECT  
    branch,  
    product_line,  
    ROUND(profit, 2) AS profit,  
    ROW_NUMBER() OVER (  
      PARTITION BY branch  
      ORDER BY profit DESC  
    ) AS rn  
  FROM product_profit  
)  
SELECT  
  branch,  
  product_line,  
  profit  
FROM ranked_profit  
WHERE rn = 1  
ORDER BY branch;
```

Result

	branch	product_line	profit
0	A	Health and beauty	-11397.97
1	B	Food and beverages	-13765.85
2	C	Home and lifestyle	-12572.17

Explanation

This query calculates profit and identifies the highest-profit product line for every branch.

Task 3: Analyzing Customer Segmentation Based on Spending

Objective

To classify customers as High, Medium, or Low spenders.

SQL Query

```
WITH customer_spending AS (  
  SELECT  
    "Customer type" AS customer_type,  
    ROUND(AVG(Total), 2) AS avg_spending  
  FROM walmart_sales  
  GROUP BY "Customer type"  
)  
SELECT  
  customer_type,  
  avg_spending,  
  CASE  
    WHEN avg_spending >= 350 THEN 'High'  
    WHEN avg_spending >= 200 THEN 'Medium'  
    ELSE 'Low'  
  END AS spending_tier  
FROM customer_spending  
ORDER BY avg_spending DESC;
```

Result

	customer_type	avg_spending	spending_tier
0	Member	327.79	Medium
1	Normal	318.12	Medium

Explanation

This query calculates average spending and categorizes customers using a CASE statement.

Task 4: Detecting Anomalies in Sales Transactions

Objective

To identify unusually high or low transactions.

SQL Query

```
WITH product_avg AS (  
SELECT  
"Product line" AS product_line,  
AVG(Total) AS avg_total  
FROM walmart_sales  
GROUP BY "Product line"  
)  
SELECT  
w."Invoice ID" AS invoice_id,  
w."Product line" AS product_line,  
ROUND(w.Total, 2) AS transaction_total,  
ROUND(p.avg_total, 2) AS average_product_total,  
CASE  
WHEN w.Total > p.avg_total * 1.5 THEN 'High Anomaly'  
WHEN w.Total < p.avg_total * 0.5 THEN 'Low Anomaly'  
ELSE 'Normal'  
END AS anomaly_status  
FROM walmart_sales w  
JOIN product_avg p  
ON w."Product line" = p.product_line  
WHERE w.Total > p.avg_total * 1.5
```

OR w.Total < p.avg_total * 0.5

ORDER BY transaction_total DESC;

Result

		invoice_id	product_line	transaction_total	average_product_total	anomaly_status
...	0	860-79-0874	Fashion accessories	1042.65	305.09	High Anomaly
	1	687-47-8271	Fashion accessories	1039.29	305.09	High Anomaly
	2	283-26-5248	Food and beverages	1034.46	322.67	High Anomaly
	3	751-41-9720	Home and lifestyle	1023.75	336.64	High Anomaly
	4	303-96-2227	Home and lifestyle	1022.49	336.64	High Anomaly
	...	...	...	...	...	...
	564	236-86-3015	Home and lifestyle	14.68	336.64	Low Anomaly
	565	192-98-7397	Fashion accessories	13.42	305.09	Low Anomaly
	566	279-62-1445	Fashion accessories	13.17	305.09	Low Anomaly
	567	308-39-1707	Fashion accessories	12.69	305.09	Low Anomaly
	568	784-21-9238	Sports and travel	10.68	332.07	Low Anomaly
569 rows x 5 columns						

Explanation

This query compares each transaction with the average transaction value of its product line.

Task 5: Most Popular Payment Method by City

Objective

To identify the most frequently used payment method in each city.

SQL Query

```
WITH payment_counts AS (  
  SELECT  
    City AS city,  
    Payment AS payment_method,  
    COUNT(*) AS usage_count  
  FROM walmart_sales  
  GROUP BY City, Payment  
)  
,  
ranked_payments AS (  
  SELECT  
    city,  
    payment_method,
```

```

usage_count,
ROW_NUMBER() OVER (
PARTITION BY city
ORDER BY usage_count DESC
) AS rn
FROM payment_counts
)
SELECT
city,
payment_method,
usage_count
FROM ranked_payments
WHERE rn = 1
ORDER BY city;

```

Result

Insert Task 5 screenshot here.

Explanation

This query counts payment usage and ranks the most popular method in each city.

Task 6: Monthly Sales Distribution by Gender

Objective

To compare monthly sales generated by male and female customers.

SQL Query

```

SELECT
strftime('%Y-%m', Date) AS month,
Gender AS gender,
ROUND(SUM(Total), 2) AS total_sales
FROM walmart_sales
GROUP BY month, Gender
ORDER BY month, Gender;

```

Result



	city	payment_method	usage_count
0	Mandalay	Ewallet	113
1	Naypyitaw	Cash	124
2	Yangon	Ewallet	126

Explanation

This query summarizes total sales by month and gender.

Task 7: Best Product Line by Customer Type

Objective

To find the highest-selling product line for each customer type.

SQL Query

```
WITH sales_by_type AS (  
  SELECT  
    "Customer type" AS customer_type,  
    "Product line" AS product_line,  
    SUM(Total) AS total_sales  
  FROM walmart_sales  
  GROUP BY "Customer type", "Product line"  
)  
,  
ranked AS (  
  SELECT  
    customer_type,  
    product_line,  
    ROUND(total_sales, 2) AS total_sales,  
    ROW_NUMBER() OVER (  
      PARTITION BY customer_type  
      ORDER BY total_sales DESC  
    ) AS rn  
  FROM sales_by_type  
)  
SELECT
```

```
customer_type,
product_line,
total_sales
FROM ranked
WHERE rn = 1
ORDER BY customer_type;
```

Result

	customer_type	product_line	total_sales
0	Member	Food and beverages	31357.62
1	Normal	Electronic accessories	29839.04

Explanation

This query identifies the product line with the highest sales for Member and Normal customers.

Task 8: Identifying Repeat Customers

Objective

To detect repeat purchases within 30 days.

SQL Query

```
WITH purchases AS (
SELECT
"Customer type" AS customer_type,
Date,
LAG(Date) OVER (
PARTITION BY "Customer type"
ORDER BY Date
) AS prev_date
FROM walmart_sales
)
SELECT
customer_type,
COUNT(*) AS repeat_purchases
FROM purchases
WHERE prev_date IS NOT NULL
```

```
AND julianday(Date) - julianday(prev_date) <= 30
```

```
GROUP BY customer_type
```

```
ORDER BY repeat_purchases DESC;
```

Result

	customer_type	repeat_purchases
0	Member	215
1	Normal	196

Explanation

This query analyzes purchasing dates to identify repeat customer behavior.

Task 9: Finding Top 5 Customers by Sales Volume

Objective

To identify the highest revenue-generating customer categories.

SQL Query

```
SELECT
```

```
"Customer type" AS customer_type,
```

```
ROUND(SUM(Total), 2) AS total_sales
```

```
FROM walmart_sales
```

```
GROUP BY "Customer type"
```

```
ORDER BY total_sales DESC
```

```
LIMIT 5;
```

Result

	customer_type	total_sales
0	Member	164223.44
1	Normal	158743.31

Explanation

This query ranks customer types based on total sales.

Task 10: Analyzing Sales Trends by Day of the Week

Objective

To determine which day of the week generates the highest sales.

SQL Query

```
SELECT
CASE strftime('%w', Date)
WHEN '0' THEN 'Sunday'
WHEN '1' THEN 'Monday'
WHEN '2' THEN 'Tuesday'
WHEN '3' THEN 'Wednesday'
WHEN '4' THEN 'Thursday'
WHEN '5' THEN 'Friday'
WHEN '6' THEN 'Saturday'
END AS day_of_week,
ROUND(SUM(Total), 2) AS total_sales
FROM walmart_sales
GROUP BY strftime('%w', Date)
ORDER BY total_sales DESC;
```

Result

	month	gender	total_sales
0	None	Female	93189.74
1	None	Male	94421.75
2	2019-01	Female	6397.93
3	2019-01	Male	3426.15
4	2019-02	Female	7477.21
5	2019-02	Male	5169.55
6	2019-03	Female	5793.65
7	2019-03	Male	6605.58
8	2019-04	Female	5491.00
9	2019-04	Male	2466.63
10	2019-05	Female	9191.71
11	2019-05	Male	3606.98
12	2019-06	Female	3290.25
13	2019-06	Male	6321.98
14	2019-07	Female	5308.93
15	2019-07	Male	6191.79
16	2019-08	Female	9795.38
17	2019-08	Male	3708.40
18	2019-09	Female	6414.48
19	2019-09	Male	7352.80
20	2019-10	Female	5433.13
21	2019-10	Male	4432.07

Explanation

This query calculates total sales for each day and identifies the top-performing day.

Key Findings

The analysis revealed differences in sales growth across branches, identified the most profitable product lines, segmented customers based on spending, highlighted payment preferences by city, and showed important sales trends by gender and day of the week.

Conclusion

This project demonstrates how SQL can be used to analyze retail sales data and uncover meaningful business insights. The findings support better decision-making in sales, marketing, and operations.

References

- [Walmart Sales Dataset](#)
- [Python Pandas Documentation](#)
- [SQLite Documentation](#)
- [Google Colab Documentation](#)